

MixBytes()

[SECURITY AUDIT REPORT]

Gearbox Protocol Integrations-v3 (Securitize RWA)

PREPARED FOR: GEARBOX

REPORT DATE

August 24, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	6
1.4 Security Audit Methodology	8
1.5 Risk Classification	10
1.6 Summary of Findings	11
2. Findings Report	11
2.1 Critical	11
2.2 High	11
2.3 Medium	12
M-1 Using the ordinary discount blocks liquidation of expired Securitize-only accounts	12
2.4 Low	14
L-1 Claims that reduce phantom collateral do not enable safe prices	14
3. About MixBytes	15

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Gearbox users hold leveraged positions through Credit Accounts. The audited suite connects those accounts to Securitize digital securities through a synchronous on-ramp, an asynchronous redemption flow, and a dedicated liquidation helper. The on-ramp exchanges an approved stablecoin for a DS token. The redemption gateway creates a dedicated redeemer clone, sends DS tokens to an external redemption account, and later lets the Credit Account claim stablecoins transferred to that clone.

While external settlement is pending, `SecuritizeRedemptionPhantomToken` stands in for the expected stablecoin proceeds and counts as Gearbox collateral until the real stablecoins are claimed. Because DS tokens and pending redeemers can only move to eligible wallets, a dedicated `SecuritizeLiquidator` lets a registered Securitize wallet take over both when an account is fully liquidated. Security therefore depends on several parts holding together at once: Gearbox health accounting and loss policy, the NAV and fee inputs that price the collateral, ownership of the redeemer clones, the ERC-4626 wrapping of the underlying stablecoin, Securitize wallet eligibility, and the off-chain settlement process.

The security auditors reviewed the integration independently and then consolidated their results. The review combined line-by-line inspection of the six in-scope contracts, threat modelling of the actors and trust boundaries, entry-point and external-call enumeration, attack-path tracing through Gearbox core and the deployed Securitize on-ramp and RedStone NAV stack, AI-assisted candidate generation with manual triage, and on-chain verification of the deployed proxies and archived NAV history; every reported issue was independently validated from source to sink.

Scope Limitations

The audit scope is limited to the six Securitize integration contracts under review. Gearbox core and Periphery, the upstream Securitize contracts, shared contracts, interfaces, tests, mocks, scripts, specifications, and deployment configuration are outside the audit scope, and were consulted only where their behavior determines whether an in-scope path is reachable or harmful.

Review Coverage

In addition to the audit team's general smart-contract security checklist, the review covered the following integration-specific questions:

- Does phantom collateral converge from the DS amount to the actual stablecoin proceeds across pending, settled, transferred, and claimed states?
- Are NAV units, snapshots, freshness, zero values, fees, caps, and rounding handled consistently by borrowing and liquidation paths?
- Can both ordinary and healthy-expired full liquidation realize every asset that the helper includes in its consideration?
- Does the public helper preserve the external caller's authorization at Gearbox loss-policy boundaries?

- Can gateway ownership, unclaimed-redeemer membership, transfer authorization, or canonical `transferMaster` wiring reallocate a claim without corresponding consideration?
- Do Securitize compliance, wallet-registration, pause, lock, and transfer restrictions preserve a liquidation or recovery path?
- Does the stablecoin-to-underlying conversion preserve a strict one-to-one asset/share relationship, and are all adapters registered against the intended targets?
- Can reentrancy, duplicate entries, zero amounts, stale records, or claim sequencing leave debt supported by removed phantom value?

Additional Attack Hypotheses

In addition to the issues described in this report, the audit examined the following protocol-specific attack hypotheses.

- The review checked whether stablecoins already transferred to a redeemer could become unreachable during liquidation. They remain recoverable because the gateway adapter can claim them into the Credit Account before liquidation completes.
- The review checked whether an early `claim` could remove phantom collateral without a health check. Gearbox recalculates account health after the claim; the narrower omission of safe prices is reported as a Low-severity finding.
- A hostile `transferMaster` moving another account's redemption. The gateway still restricts the transfer source to the redeemer owner, and Gearbox's final collateral check prevents a successful liquidation from leaving unsupported debt.
- Whether a reused Credit Account address could let a new borrower inherit a prior borrower's orphaned redeemer. The account factory the suite requires deploys a fresh Credit Account for every opening, so an address is never reissued to another borrower.
- Whether a stablecoin donation could inflate the helper's liquidity gate and leave a genuinely unhealthy account unliquidatable. An ordinary liquidation can claim that stablecoin into the Credit Account through the gateway adapter and realize it, so the counted liquidity is reachable rather than stranded.

Additional Notes

- The liquidation helper values a pending claim from current NAV without the starting-NAV cap used by the phantom token. The direction of any difference depends on the external settlement-NAV policy, which was not established.
- The deployed NAV providers read push-fed RedStone values. A valid update before an operation supplies the current positive value; after 30 hours without an update, the adapter reverts rather than returning zero or indefinitely stale data.
- The liquidation helper wraps the liquidator's contribution of the wrapper's asset into the underlying ERC-4626 just before adding it and treats the deposited asset amount as the resulting share amount, which is exact only for a wrapper that converts one-to-one; the Gearbox RWA-underlying interface requires that conversion, so a non-one-to-one wrapper is a deployment constraint rather than an in-scope exploit.
- When the redemption stablecoin differs from the wrapper's asset, the liquidation helper prices that stablecoin through the price oracle and moves it to the liquidator with a collateral withdrawal, so the stablecoin must be registered both in the price oracle and as a Credit Manager collateral token.
- Each new redemption makes unconditional external calls to the Securitize whitelister and the redemption logger. If either is paused or mis-registered, new redemptions revert until it is restored, while existing positions remain valued and claimable.

Overall, the codebase is compact and separates the on-ramp, redemption, and liquidation responsibilities clearly. Its main architectural choices are acceptable and follow from the specifics of the integrated Securitize product, whose settlement is asynchronous and off-chain and whose DS transfers are gated by wallet eligibility: a dedicated redeemer clone per redemption, a phantom token that represents pending proceeds as collateral, and a compliant liquidation helper that opens a transfer window for unsettled redeemers.

After the re-audit, the client updated `SecuritizeLiquidator` in commit

`744a4f64a3fd711768ddce55d190e04513cd36f2` to support a redemption stablecoin that differs from the asset of the ERC-4626 underlying: when the tokens differ, the account's stablecoin balance joins the liquidator's consideration at its oracle price, while the equal-token configuration keeps its previous behavior. The audit team reviewed the update; no new security issues were identified in the reviewed changes, and the deployment constraint the update introduces is recorded in the additional notes.

1.3 Project Overview

Summary

CLIENT	Gearbox Protocol
CATEGORY	Lending
PROJECT	integrations-v3 (Securitize RWA)
TYPE	Solidity
PLATFORM	EVM
TIMELINE	10.08.2026 – 24.08.2026

Scope of Audit

```
contracts/integrations/securitize/SecuritizeLiquidator.sol
```

```
contracts/integrations/securitize/SecuritizeOnRampAdapter.sol
```

```
contracts/integrations/securitize/SecuritizeRedeemer.sol
```

```
contracts/integrations/securitize/SecuritizeRedemptionGateway.sol
```

```
contracts/integrations/securitize/SecuritizeRedemptionGatewayAdapter.sol
```

```
contracts/integrations/securitize/SecuritizeRedemptionPhantomToken.sol
```

Versions Log

DATE	COMMIT HASH	NOTE
10.08.2026	e56429547af7f9452d5d552b419d07079d2695c4	Initial Commit
14.08.2026	22754fff8278c5338e5ff0249d9c37d4680186f4	Re-audit Commit
24.08.2026	744a4f64a3fd711768ddce55d190e04513cd36f2	SecuritizeLiquidator Update Commit

Mainnet Deployments

Deployment verification will be conducted via <https://bcr.gearbox.finance/>

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

ID	FINDING	SEVERITY	STATUS
M-1	Using the ordinary discount blocks liquidation of expired Securitize-only accounts	● MEDIUM	● FIXED
L-1	Claims that reduce phantom collateral do not enable safe prices	● LOW	● FIXED

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

M-1 Using the ordinary discount blocks liquidation of expired Securitize-only accounts

● MEDIUM

● FIXED

DESCRIPTION

A Securitize-enabled market can use an expirable Credit Facade: Core exposes an `expirable` constructor flag and lets the configurator set an expiration date ([CreditFacadeV3.sol:171-187](#), [CreditFacadeV3.sol:841-854](#)). The Credit Manager constructor rejects an expired liquidation premium above the ordinary premium and calculates each discount as 100% minus the corresponding premium ([CreditManagerV3.sol:174-189](#)). Therefore, `liquidationDiscountExpired` is always at least as high as `liquidationDiscount`; whenever the two values differ, the expired discount is necessarily the higher one.

[SecuritizeLiquidator.sol:74-110](#) nevertheless always uses the ordinary `liquidationDiscount` to calculate the underlying supplied by the liquidator. Core uses the higher `liquidationDiscountExpired` when the account is liquidated after the Credit Facade expires ([CreditFacadeV3.sol:326-362](#), [CreditManagerV3.sol:306-346](#)). The helper therefore supplies less underlying than Core requires for the same liquidation.

The helper removes the DS-token balance and transfers any pending redeemers while adding only the ordinary-discounted `underlyingAmount` ([SecuritizeLiquidator.sol:156-195](#)). Core then reverts whenever that contribution and the collateral retained on the account are insufficient to satisfy its expired-discounted minimum remaining funds ([CreditManagerV3.sol:341-357](#)). A Securitize-only account guarantees that no unrelated collateral can cover the difference, but it is not a prerequisite: any account whose retained collateral is smaller than the shortfall is blocked. The assets do not move because the transaction is atomic, while the expired account remains unliquidated. Health-based liquidations are unaffected.

The impact is Medium because an expired position cannot be closed permissionlessly and restoring liquidatability requires governance to change the helper or the configuration; even that config workaround is not neutral, since equalizing the discounts raises the expired premium and thus the loss on every expired liquidation. The likelihood is Medium because the triggering conditions are the expected operating state rather than an edge case: pricing expired liquidations at a lower premium is standard, and a trusted liquidator meets the revert in ordinary operation. The resulting severity is Medium.

RECOMMENDATION

We recommend taking the liquidator's underlying contribution as a call parameter, as Core's native liquidation does, and letting Core's remaining-funds guard enforce the floor, since overpayment is returned to the account (`to == creditAccount`), a rational liquidator supplies exactly what Core requires. Failing that, compute the amount with `liquidationDiscountExpired` for an expired liquidation and `liquidationDiscount` otherwise.

CLIENT'S COMMENTARY

Fixed in `22754fff8278c5338e5ff0249d9c37d4680186f4` . Collateral and liquidation values now use the normal discount when the account is unhealthy, and expired discount otherwise.

2.4 Low

L-1 Claims that reduce phantom collateral do not enable safe prices

● LOW

● FIXED

DESCRIPTION

`SecuritizeRedemptionGatewayAdapter.sol:87-90` returns `false` after claiming the available stablecoins. The claim can reduce the account's phantom collateral because it transfers the redeemer's current balance, clears the pending DS amount, and removes the redeemer from the unclaimed set (`SecuritizeRedeemer.sol:97-101`, `SecuritizeRedemptionGateway.sol:116-124`).

The final collateral check still runs, but `CreditFacadeV3` enables safe prices and strict forbidden-token checks only when an adapter returns `true` (`CreditFacadeV3.sol:621-649`, `CreditFacadeV3.sol:818-832`). As a result, a claim can pass at main-feed prices even when the resulting account would fail at the lower of its main and reserve prices (`PriceOracleV3.sol:193-202`). This is not a complete collateral-check bypass; it affects only accounts for which the normal and conservative checks produce different results.

RECOMMENDATION

We recommend returning `true` from `claim` so that every redemption claim enables safe prices and strict forbidden-token checks. A regression test should cover a claim that passes at main-feed prices but fails with the configured reserve feed.

CLIENT'S COMMENTARY

Fixed in `22754fff8278c5338e5ff0249d9c37d4680186f4`.

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io

[!\[\]\(488d36215f31304317ffb20d512ebb61_img.jpg\)](#) [!\[\]\(1496aacae6d389f910d5c8bbde8ccdd1_img.jpg\)](#) [!\[\]\(d8585384138061be445a6f30ccb50635_img.jpg\)](#) [!\[\]\(e35bba9aea99daef87ead1a8ef656c9d_img.jpg\)](#)